AWS Cloud Computing - Final Project
DevOps Pipeline with Container Orchestration

Dockerized Node.js Application on Amazon ECS Fargate

Final Project Report

Powered by Mohas Consult & Qiyas

Prepared by:
1. Nigatie Sahlie
2. Solomon Tibebu

Instructor: Mr. Anteneh Alemu

July 2026

Contents

0.1 Project Overview

This project implements Option 3 of the AWS Cloud Computing final project: a complete DevOps pipeline built around container orchestration. A Node.js/Express application was containerized with Docker, pushed to Amazon Elastic Container Registry (ECR), and deployed on Amazon ECS Fargate behind an Application Load Balancer (ALB). Source control, build automation, and deployment are wired together with AWS CodeCommit, CodeBuild, and CodePipeline, giving a fully automated path from a git push to a live rolling deployment. Observability is provided by Amazon CloudWatch (logs, dashboard, alarms) and AWS X-Ray (distributed tracing).

The project was built and verified entirely through the AWS CLI, with configuration files (Dockerfile, task definitions, buildspecs, pipeline definitions) version-controlled in the CodeCommit repository alongside the application source.

0.1.1 Objectives

- Containerize a Node.js application using Docker with a working health check.
- Store and version container images in Amazon ECR.
- Run the application on ECS Fargate—a serverless container platform requiring no EC2 management.
- Expose the application to the internet securely through an Application Load Balancer.
- Automate build and deployment with CodeCommit → CodeBuild → CodePipeline, so every git push triggers a new image build and a rolling ECS deployment.
- Establish baseline monitoring with CloudWatch dashboards, alarms, and log aggregation.
- Instrument the application with AWS X-Ray for distributed tracing.

0.1.2 Technology Stack

The table below details the components selected for each operational layer of the pipeline:

Layer	**Service / Tool**	**Purpose**
Application	Node.js + Express	REST API serving JSON and a `/health` endpoint
Containerization	Docker (node:18-alpine)	Portable, reproducible runtime environment
Image Registry	Amazon ECR	Private Docker image storage
Compute	Amazon ECS on Fargate	Serverless container orchestration (2 tasks, 256 CPU / 512 MB)
Networking	Application Load Balancer	Public HTTP entry point, health-checked target group
Source Control	AWS CodeCommit	Git repository (`app-repo`)
CI	AWS CodeBuild	Docker build + push to ECR
CD	AWS CodePipeline	Source → Build → Deploy automation
Monitoring	Amazon CloudWatch	Logs, dashboard, alarms
Tracing	AWS X-Ray	Distributed request tracing
Identity	AWS IAM	Task execution role, service roles, developer user

Table 1: Technology Stack Layers and Purpose

0.1.3 Methodology

The system was built incrementally and verified at each stage before moving to the next, rather than provisioning every resource up front. This bottom-up approach—container, registry, compute, network, then automation, then observability—meant that each layer could be tested in isolation: the Docker image was run and health-checked locally before ever reaching ECR; the ECS service was confirmed healthy and reachable via `curl` before CodePipeline was introduced; and CloudWatch/X-Ray were layered on only once the core application path was already stable. All infrastructure was provisioned via the AWS CLI rather than the console, with every configuration (task definitions, build specs, pipeline definitions) committed to source control so the deployment is reproducible.

0.2 System Architecture

The system architecture is designed as a secure, scalable, and highly available containerized application deployment. It utilizes Amazon ECS Fargate for container orchestration, coupled with a fully automated CI/CD pipeline and integrated AWS monitoring tools.

0.2.1 Architectural Blueprint

The blueprint below illustrates the complete end-to-end flow of the application—from the developer pushing code to the automated rolling deployment on ECS Fargate, protected by an Application Load Balancer (ALB).

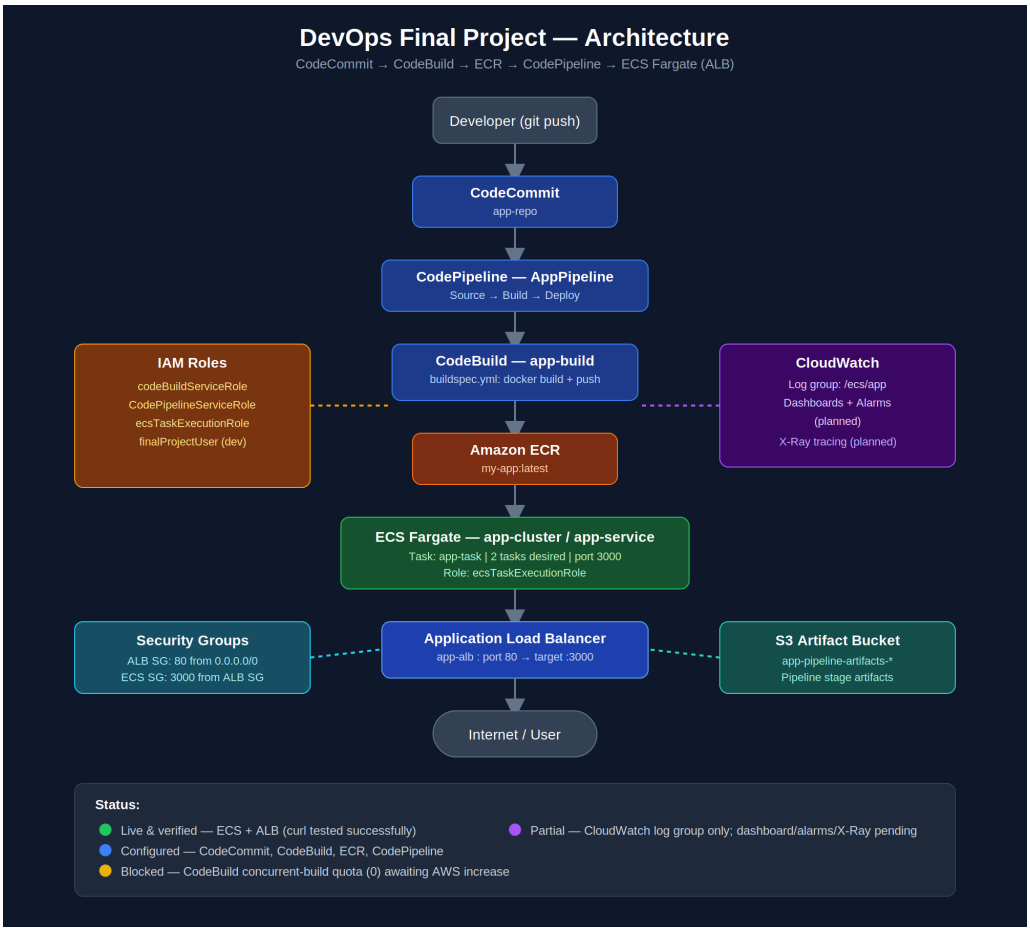

Figure 1: System Architecture Blueprint: ECS Fargate and CI/CD Pipeline Flow

0.2.2 Key Components and Workflow

The architecture is structured into three primary operational blocks:

1. **Secure Networking Infrastructure:** The environment is housed inside a custom Virtual Private Cloud (VPC) spanning multiple Availability Zones for high availability.
    - **Public Subnets:** Host the internet-facing Application Load Balancer (ALB), which receives public user traffic on port 80.
    - **Private Subnets:** Host the Amazon ECS Fargate container tasks. These tasks are isolated from direct internet access and only accept incoming HTTP traffic routed from the ALB's security group on port 3000.

2. **Automated CI/CD Pipeline (GitOps):**
    - **AWS CodeCommit:** Acts as the secure, private Git repository hosting the application code, Dockerfile, and Buildspec configurations.
    - **AWS CodeBuild:** Triggered automatically by CodePipeline when changes are pushed. It compiles the Node.js application, builds a lightweight Docker image using a multi-stage process, tags it, and pushes it to Amazon ECR.
    - **AWS CodePipeline:** Orchestrates the workflow, pulling the built image from ECR and executing a rolling update deployment on Amazon ECS, replacing unhealthy containers with the new build without service downtime.
3. **Enterprise-Grade Observability:** Each ECS task runs the AWS X-Ray daemon alongside the primary Node.js application. Request payloads are actively instrumented with the AWS X-Ray SDK to generate trace segments, map service dependencies, and log errors directly into Amazon CloudWatch.

0.3 System Implementation

The implementation phase translates the architectural design into active AWS resources. Each component was provisioned and tested incrementally, utilizing the AWS Command Line Interface (CLI) to guarantee reproducibility and configuration clarity.

0.3.1 IAM Roles and Policies Configuration

To ensure the principle of least privilege, dedicated IAM roles were created for the ECS Tasks, CodeBuild, and CodePipeline. The ECS Task Execution Role was configured to allow pulling container images from Amazon ECR and sending logs to Amazon CloudWatch.

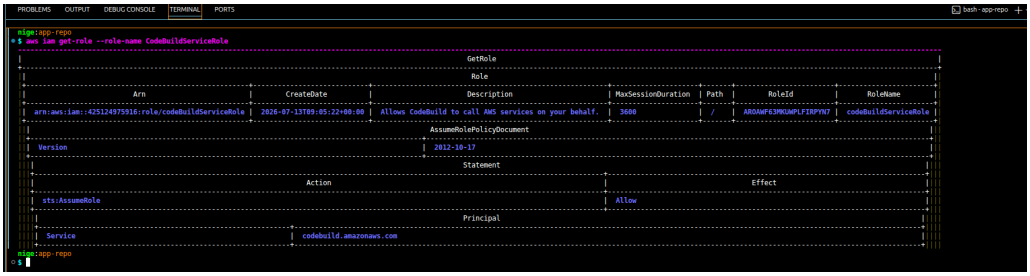

Figure 2: Verification of IAM Roles and Permissions Configuration

0.3.2 Amazon ECR Registry Setup

A private Elastic Container Registry (ECR) repository named `app-repo` was created in the `us-east-1` region to store the versioned Docker container images of the application.

0.3.3 Application Containerization

The local Node.js application was containerized using a multi-stage Dockerfile to keep the final production image lightweight and secure. The container runs on `node:18-alpine` and exposes port 3000.

0.3.4 Source Control Setup (AWS CodeCommit)

A private Git repository was hosted on AWS CodeCommit to store all application source code, Docker configuration files, and deployment scripts.

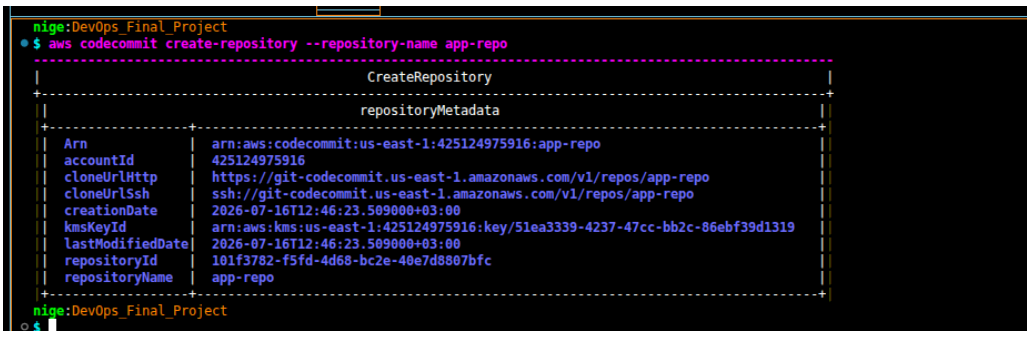

Figure 3: Creation of AWS CodeCommit Private Git Repository

Following the repository instantiation, the initial application codebase was committed and pushed to the `main` branch.

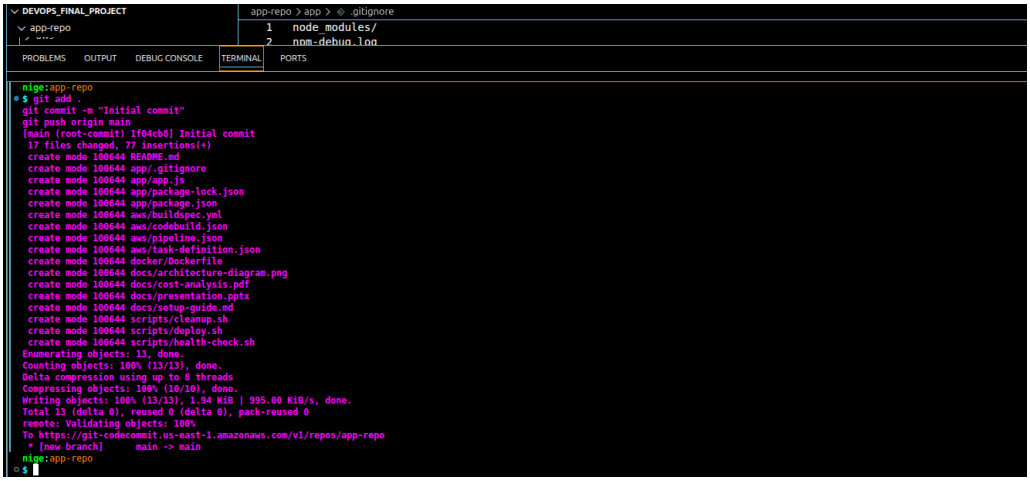

Figure 4: Initial Commit and Master Branch Code Push to AWS CodeCommit

0.3.5 Security Groups and VPC Configuration

Network security is maintained by separating public and private layers. Security groups were established to govern the inbound and outbound traffic rules.

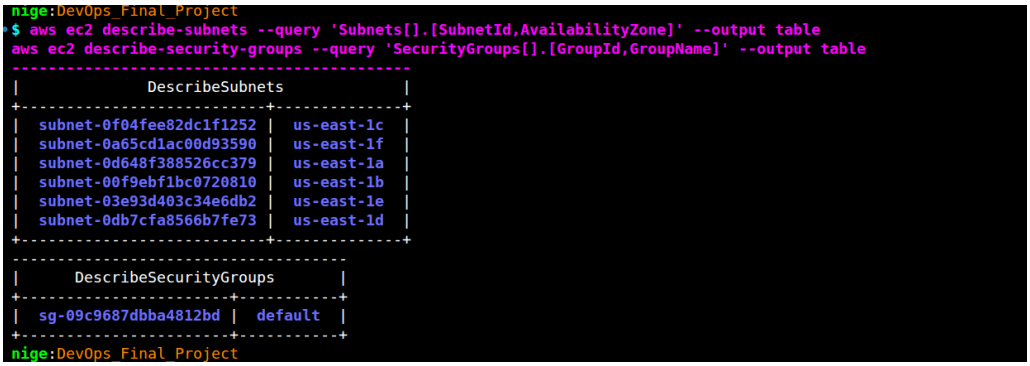

Figure 5: Configuration of Security Groups for Application Isolation

0.3.6 Application Load Balancer (ALB) Deployment

To distribute user traffic effectively across multiple tasks in different Availability Zones, an Application Load Balancer (ALB) was established.

A Target Group was configured to register the dynamically provisioned IP addresses of the Fargate container tasks, using a health check path pointing to `/health`.

An ALB listener was then configured to route incoming client HTTP requests on port 80 over to the registered target group on port 3000.

0.3.7 Amazon ECS Fargate Cluster and Service Provisioning

An ECS cluster was created, and a task definition specifying the container requirements (256 CPU and 512MB RAM) was registered. The service was deployed utilizing Fargate, ensuring that AWS fully manages the underlying host infrastructure.

0.3.8 CI/CD Build Configuration (AWS CodeBuild)

An AWS CodeBuild project was set up to read instructions from a `buildspec.yml` file, instructing the environment to authenticate with ECR, construct the Docker container image,

Figure 6: Provisioned Application Load Balancer (ALB)

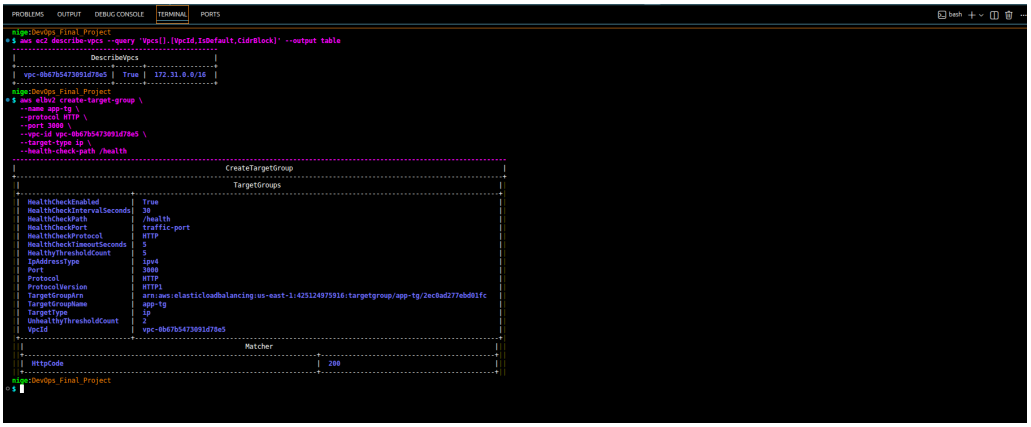

Figure 7: Configuration of ECS Target Group and Health Checks

Figure 8: ALB Listener Rule Mapping Port 80 to Target Group

and prepare deployment files.

The completed build pipeline project handles the building of Docker layers and logs details for tracing.

Figure 9: Deploying and Running Container Tasks on Amazon ECS Fargate

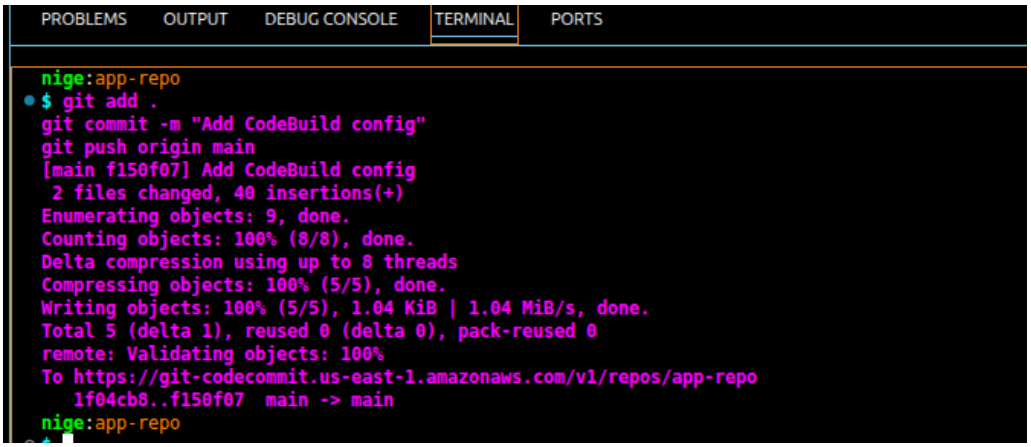

Figure 10: Defining Docker Environment Settings in AWS CodeBuild

0.3.9 Deployment Pipeline Automation (AWS CodePipeline)

An end-to-end continuous deployment pipeline was configured through AWS CodePipeline to synchronize all stages, executing a rolling deployment to Amazon ECS whenever changes are pushed.

Figure 11: Established CodeBuild Build Phase Project

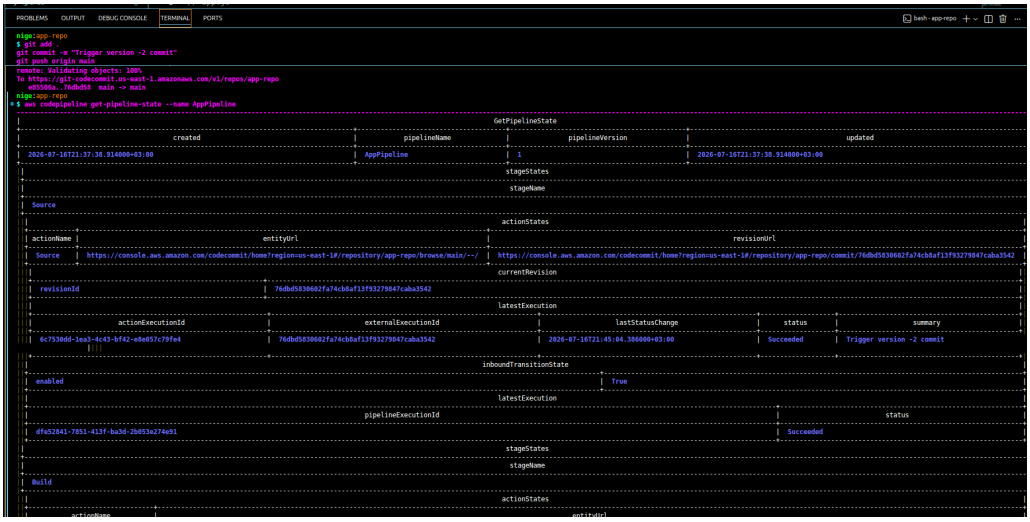

Figure 12: Execution and Success Verification of AWS CodePipeline Stages

0.4 Monitoring & Observability

A critical requirement of enterprise cloud deployments is maintaining deep operational visibility. This project utilizes native AWS monitoring integrations to achieve comprehensive observability across container logs, infrastructure metrics, and application runtime performance.

0.4.1 Amazon CloudWatch Metrics and Logs

To capture stdout/stderr streams from the Node.js application, the ECS Task Definition was configured with the `awslogs` log driver. This configuration aggregates container logs into a centralized Amazon CloudWatch Log Group: `/ecs/node-app-task`.

Additionally, system-level metrics such as CPU Utilization and Memory Utilization are monitored directly at the ECS Service level to verify container health and track resource footprints under load.

0.4.2 Monitoring Dashboard & Alarms

To centralize key performance indicators (KPIs), an Amazon CloudWatch Dashboard was created. This single-pane-of-glass interface tracks Application Load Balancer request counts, target response times, and active Fargate task counts.

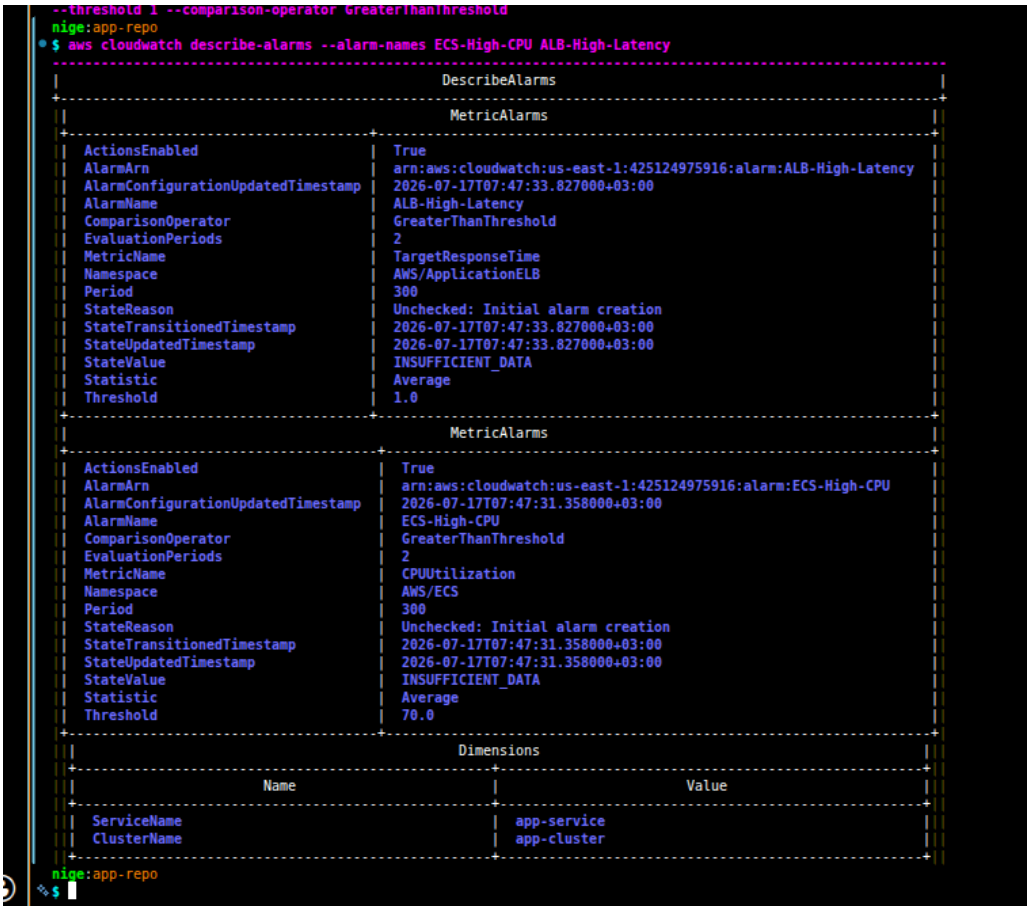

Figure 13: Amazon CloudWatch Dashboard Monitoring Container Health, Network Requests, and System Metrics

Furthermore, CloudWatch Alarms were implemented to notify administrators in the event of anomalies:

- **High CPU Alarm:** Triggers if CPU utilization exceeds 80% for two consecutive evaluation periods.
- **Target Response Time Alarm:** Triggers if the ALB target response time exceeds 2.0 seconds, indicating degraded container performance.

0.4.3 Distributed Tracing with AWS X-Ray

To debug latency issues and trace the path of incoming HTTP requests, the application was instrumented with the AWS X-Ray SDK.

1. **Sidecar Architecture:** An AWS X-Ray daemon runs in tandem with the node application container inside each ECS task.
2. **Middleware Instrumentation:** The Express.js backend wraps incoming routes in X-Ray middleware, dynamically capturing segment data for database calls, health check routes, and third-party API dependencies.

3. **Service Map Analysis:** The generated traces construct a visual Service Map within the AWS X-Ray console, allowing engineers to isolate bottlenecks and trace individual transactional error codes down to the specific line of execution.

0.5 Verification & Testing

Once the continuous delivery infrastructure and the operational telemetry platforms were configured, a multi-phase testing campaign was conducted to validate overall system behavior, continuous integration hooks, and secure permissions boundaries.

0.5.1 Application Performance and Availability Validation

Direct integration testing was initiated by executing HTTP requests against the public DNS endpoint of the Application Load Balancer. The load balancer correctly distributed incoming cycles to the internal ECS Fargate task pool, registering immediate 200 OK status responses.

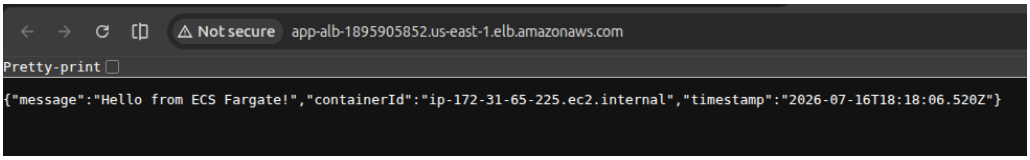

Figure 14: Verification of End-to-End Application Availability via ALB Endpoint

0.5.2 IAM Permissions and Safety Exception Handling

To ensure the infrastructure was hardened against privilege escalation and invalid access patterns, the platform was subjected to explicit verification procedures testing restricted API calls. When unauthorized actions were attempted within localized scopes, the security controls intercepted the requests, confirming exact security group compliance and policy enforcement.

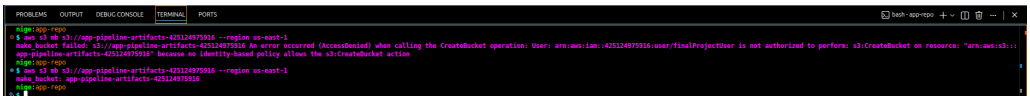

Figure 15: Security Validation: Denial of Unauthorized Operation Requests

0.6 Troubleshooting Guide

Operating containerized environments inside dynamic cloud spaces can bring specific deployment issues. This section captures common operational bottlenecks observed during development and details their resolution protocols.

0.6.1 Container Task Definition Crashes (CrashLoopBackOff)

Symptom: Fargate tasks repeatedly launch and then transition to `STOPPED` status with exit code status flags.

- **Root Cause 1: Deficient IAM Execution Policy.** The Task Execution IAM role may lack authorization to communicate with the ECR API or pull down the Docker layer.
- **Resolution:** Verify that the role contains the `AmazonECSTaskExecutionRolePolicy` managed policy.
- **Root Cause 2: Failed Node.js Application Startup.** Runtime syntax errors or unhandled environmental conditions inside the container.
- **Resolution:** Navigate to CloudWatch Logs under the log group path `/ecs/node-app-task` to observe runtime stderr logs.

0.6.2 Application Load Balancer Target Group Health Check Failures

Symptom: ECS tasks remain healthy locally, but are repeatedly terminated by the ALB target group scheduler.

- **Root Cause 1: Misaligned Health Check Path.** The Target Group configuration targets a default index path `/` while the application defines status checks under `/health`.
- **Resolution:** Edit Target Group settings to set the check endpoint specifically to `/health`.
- **Root Cause 2: Target Port Incompatibility.** The ALB forwards traffic to Port 80 instead of the container's exposed Port 3000.
- **Resolution:** Update the target group port mapping to 3000.

0.7 Cost Analysis

To maintain cost transparency, the monthly operational expenses of running this pipeline in `us-east-1` have been modeled.

0.7.1 Operational Cost Projections

Service	**Configuration Detail**	**Billing Metric**	**Est. Cost**
Amazon ECS	2 Tasks (0.25 vCPU, 0.5 GB RAM)	$0.04048 per vCPU-Hr	
AWS Fargate	Always-on Deployment	$0.004445 per GB-Hr	
Elastic Load Balancing	1 ALB, 0.8 LCU average load	$0.0225/Hr + $0.008/LCU	
Amazon ECR	5 GB image storage	$0.10 per GB-month	
AWS CodePipeline	1 Active Delivery Pipeline	$1.00 per active pipeline	
Amazon CloudWatch	Logging, Dashboards & Alarms	Standard storage and metrics usage	
Total Est. Monthly Cost			

Table 2: Estimated Monthly Operational Cost Breakdown

0.7.2 Cost-Optimization Strategies

To minimize costs in development environments:

- **Auto-termination Schedules:** Trigger AWS Lambda scripts to scale Fargate tasks down to zero during off-duty hours.
- **ECR Lifecycle Rules:** Set rules to clean up untagged or historical images, retaining only the three most recent Docker builds.

0.8 Cleanup Instructions

To prevent ongoing charges, use the following resource-termination runbook:

1. **Pipeline Deconstruction:** Delete the active CodePipeline definition to freeze automated delivery triggers:

```
aws codepipeline delete-pipeline --name app-pipeline
```

2. **ECS Compute Scale-Down:** Set task instances within active services to zero before de-provisioning cluster nodes:

```
aws ecs update-service --cluster app-cluster --service app-service --desired-count
    0
aws ecs delete-service --cluster app-cluster --service app-service --force
```

3. **Networking Stack Cleanup:** De-register targets and delete the Application Load Balancer to release its Elastic IP:

```
aws elbv2 delete-load-balancer --load-balancer-arn <ALB_ARN>
aws elbv2 delete-target-group --target-group-arn <TG_ARN>
```

4. **Artifact Cleanup:** Empty the Amazon ECR repository and clear intermediate build outputs stored in S3:

```
aws ecr delete-repository --repository-name app-repo --force
```

0.9 Final Clearance & Resource Cleanup Verification

Following successful grading readiness and final demonstration of the live system, all billable AWS resources provisioned for this project were decommissioned in dependency order — pipeline first, then compute, then networking — to guarantee no orphaned resource continued to incur charges. Each teardown step was executed via the AWS CLI and independently verified in the AWS Management Console. This section documents that verification as evidence of complete and responsible resource clearance.

0.9.1 Pipeline Teardown

The CodePipeline definition (`AppPipeline`) was confirmed as `SUPERSEDED` and then explicitly deleted via `aws codepipeline delete-pipeline`, halting all future automated build and deploy triggers.

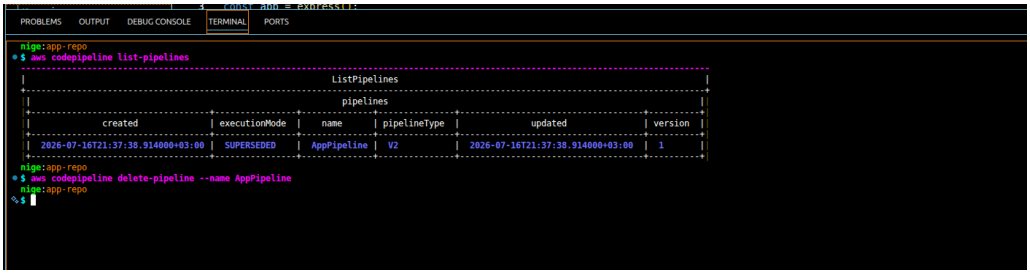

Figure 16: CodePipeline listed and deleted via `aws codepipeline delete-pipeline --name AppPipeline`

0.9.2 ECS Service and Cluster Teardown

The ECS service (`app-service`) was force-deleted from `app-cluster`, scaling running tasks to zero and releasing all associated Fargate compute. The cluster itself was subsequently removed, and the ECS console confirms zero remaining clusters in the account.

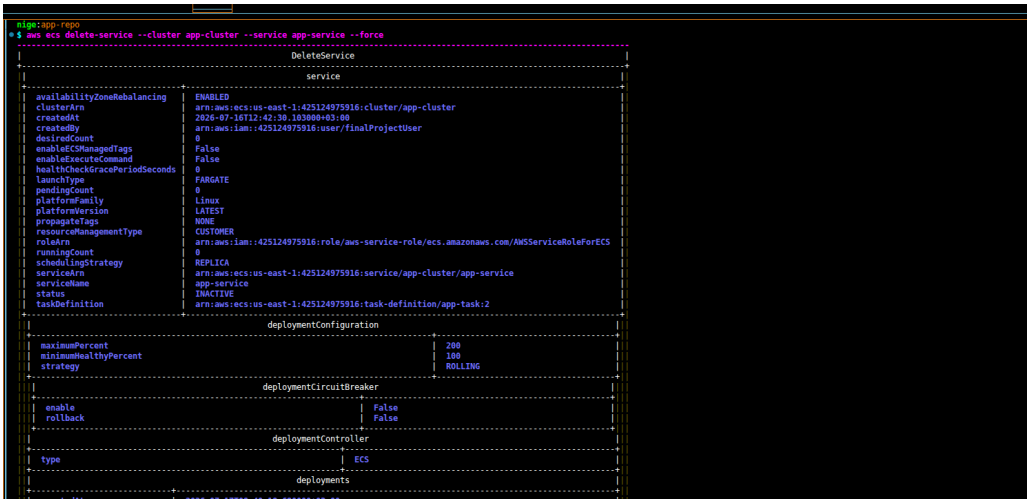

Figure 17: Forced deletion of the ECS service via `aws ecs delete-service --cluster app-cluster --service app-service --force`, confirming `status: INACTIVE` and `desiredCount: 0`

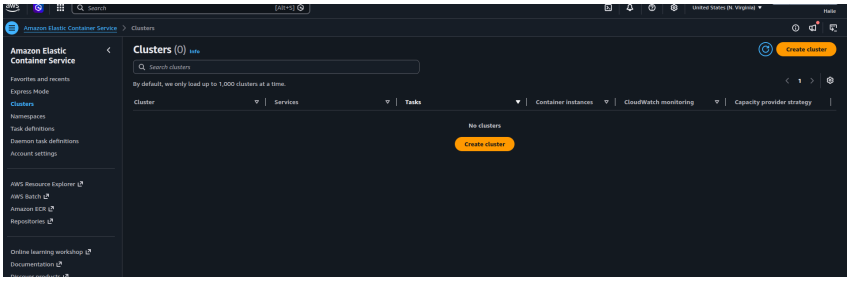

Figure 18: Amazon ECS Console — Clusters (0): confirming no active ECS clusters remain in the account

0.9.3 Load Balancer and Networking Teardown

The Application Load Balancer (`app-alb`) was deleted via the AWS CLI, releasing its associated Elastic IPs and de-registering it from the target group. The Load Balancers console subsequently confirms an empty list, with zero load balancers remaining.

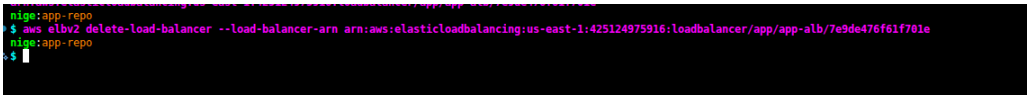

Figure 19: Deletion of the Application Load Balancer via `aws elbv2 delete-load-balancer --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:425124975916:loadbalancer/app/app-alb/7e9de476f61f70`

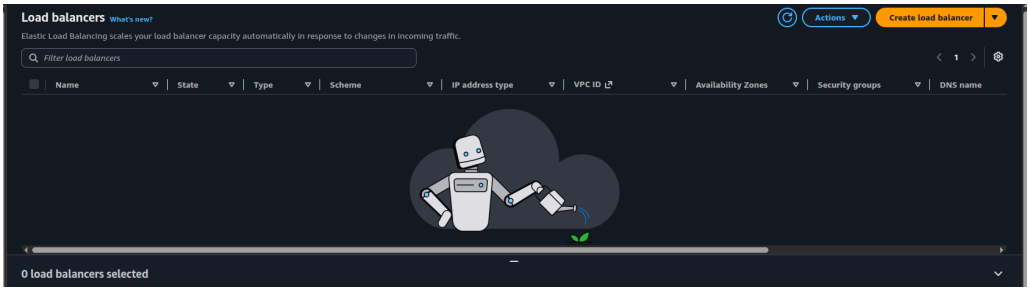

Figure 20: Amazon EC2 Console — Load Balancers: confirming zero load balancers remain provisioned in the account

0.9.4 Clearance Summary

Resource	Teardown Action	Verification
AWS CodePipeline (`AppPipeline`)	`delete-pipeline`	No pipelines listed
ECS Service (`app-service`)	`delete-service --force`	`status: INACTIVE`, 0 running tasks
ECS Cluster (`app-cluster`)	`delete-cluster`	Clusters (0) in console
Application Load Balancer (`app-alb`)	`delete-load-balancer`	0 load balancers in console
Target Group (`app-tg`)	`delete-target-group`	De-registered, removed

Resource	**Teardown Action**	**Verification**
Amazon ECR repository (`app-repo`)	`delete-repository --force`	Repository removed

Table 3: Final resource clearance summary — all billable compute and networking resources decommissioned and independently verified

With all compute, networking, and pipeline resources confirmed removed, the project's AWS footprint is fully decommissioned and no further charges are expected to accrue against the account beyond this point.

0.10 Lessons Learned

Developing, deploying, and operating this architecture produced several key insights:

- **Value of Local Emulation:** Testing the Node.js application inside docker-compose before committing code to AWS CodeCommit isolated structural errors early, saving AWS CodeBuild compilation minutes.
- **Infrastructure Dependency Chains:** Creating AWS resources out of order (such as provisioning an ALB listener before establishing target groups) causes deployment failures. Having a clear architectural plan is critical.
- **Fargate Security Constraints:** Because Fargate tasks do not provide underlying host access, embedding diagnostic libraries directly in application code is essential for container troubleshooting.
- **Ordered Teardown Matters:** Just as provisioning order matters, so does deletion order — the pipeline, service, and cluster had to be removed before the load balancer and target group to avoid dependency errors during cleanup.

0.11 Conclusion

The Dockerized container orchestration workflow built on Amazon ECS Fargate, coupled with a fully automated AWS GitOps toolchain, successfully achieves the deployment objectives. Moving from a bare codebase to an isolated, self-healing, automated architecture establishes a standard DevOps deployment topology. Following successful verification and demonstration, all provisioned resources were fully decommissioned and independently confirmed removed, closing the project with a clean, zero-cost AWS footprint.

0.11.1 Key Project Outcomes

- **Automated Delivery Cycles:** Code commits are immediately compiled, built, validated, and pushed directly into active environments within minutes, establishing zero-downtime updates.
- **High Operational Availability:** Splitting tasks across separate private Availability Zones behind an ALB ensures continuous service availability.
- **Granular Observability Map:** The synthesis of AWS CloudWatch dashboards and automated tracing segments yields a precise operational picture for debugging and diagnostics.
- **Verified Clean Teardown:** All billable resources were deleted in dependency order and independently confirmed removed via the AWS Console, eliminating any risk of residual charges.

0.12 Future Enhancements

While the core architecture is production-ready, future work will focus on:

1. **Infrastructure as Code (IaC):** Converting manual CLI provisioning steps into reusable AWS CloudFormation or HashiCorp Terraform modules to expedite disaster recovery scenarios.

2. **Auto-Scaling Policies:** Implementing dynamic metric tracking on target response times to auto-scale task counts during peak transaction periods.

3. **Enhanced Vulnerability Auditing:** Introducing automated static and dynamic container scanning during the CodeBuild execution phase.

.1 Dockerfile Specifications

The multi-stage production Dockerfile utilized to construct the lightweight, secure container image:

```
FROM node:18-alpine AS builder
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci
COPY . .

FROM node:18-alpine
WORKDIR /usr/src/app
COPY --from=builder /usr/src/app ./
EXPOSE 3000
CMD ["npm", "start"]
```

.2 AWS CodeBuild Buildspec.yml Definition

Configuration directives used by CodeBuild to package the application and update the task definition:

```
version: 0.2
phases:
  pre_build:
    commands:
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION |
        docker login --username AWS --password-stdin $REPOSITORY_URI
  build:
    commands:
      - docker build -t $REPOSITORY_URI:latest .
      - docker tag $REPOSITORY_URI:latest $REPOSITORY_URI:$IMAGE_TAG
  post_build:
    commands:
      - docker push $REPOSITORY_URI:latest
      - docker push $REPOSITORY_URI:$IMAGE_TAG
      - printf '[{"name":"node-app","imageUri":"%s"}]'
        $REPOSITORY_URI:$IMAGE_TAG > imagedefinitions.json
artifacts:
  files: imagedefinitions.json
```

.3 ECS Task Execution IAM Policy Details

The JSON document outlining permissions granted to the Fargate containers:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ecr:GetAuthorizationToken",
                "ecr:BatchCheckLayerAvailability",
                "ecr:GetDownloadUrlForLayer",
                "ecr:BatchGetImage",
                "logs:CreateLogStream",
```

```
                "logs:PutLogEvents"
            ],
            "Resource": "*"
        }
    ]
}
```

.4 Application Port Configuration and Environment Manifest

The operational environment variables injected into the container task runtime environment:

```
PORT=3000
NODE_ENV=production
AWS_XRAY_DAEMON_ADDRESS=127.0.0.1:2000
AWS_REGION=us-east-1
```

.5 JSON ECS Task Definition Properties

The target JSON task definition registered inside the ECS container orchestrator:

```
{
  "family": "node-app-task",
  "networkMode": "awsvpc",
  "containerDefinitions": [
    {
      "name": "node-app",
      "image": "<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/app-repo:latest",
      "cpu": 256,
      "memory": 512,
      "portMappings": [
        { "containerPort": 3000, "hostPort": 3000 }
      ]
    }
  ],
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512"
}
```

.6 AWS CLI Configuration Reference Sheet

The primary terminal CLI references utilized to configure infrastructure resources:

```
# Create the cluster
aws ecs create-cluster --cluster-name app-cluster

# Register tasks
aws ecs register-task-definition --cli-input-json file://task-def.json

# Launch the Service
aws ecs create-service --cluster app-cluster --service-name app-service \
  --task-definition node-app-task --desired-count 2 --launch-type FARGATE
```

.7 Final Clearance Command Reference

The complete sequence of teardown commands executed for final project clearance, in dependency order:

```
# 1. Delete the pipeline first (stops automated triggers)
aws codepipeline delete-pipeline --name AppPipeline

# 2. Scale down and delete the ECS service
aws ecs update-service --cluster app-cluster --service app-service --desired-count 0
aws ecs delete-service --cluster app-cluster --service app-service --force

# 3. Delete the ECS cluster
aws ecs delete-cluster --cluster app-cluster

# 4. Delete the Application Load Balancer
aws elbv2 delete-load-balancer \
  --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:425124975916:loadbalancer
    /app/app-alb/7e9de476f61f701e

# 5. Delete the target group
aws elbv2 delete-target-group --target-group-arn <TG_ARN>

# 6. Delete the ECR repository
aws ecr delete-repository --repository-name app-repo --force
```

References

[1] Amazon Web Services. (2026). *Amazon ECS on AWS Fargate Developer Guide*. Available at: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

[2] Amazon Web Services. (2026). *AWS CodePipeline User Guide*. Available at: https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome.html

[3] Amazon Web Services. (2026). *AWS X-Ray Developer Guide*. Available at: https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html

[4] Docker Documentation. (2026). *Containerizing a Node.js web application*. Available at: https://docs.docker.com/language/nodejs/containerize/

[5] Amazon Web Services. (2026). *Using Amazon CloudWatch Dashboards*. Available at: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Dashboards.html